

OSSP Skill Week 8 - Combined Redirection and Background Jobs

Task 1: Combined Redirection

Aim

To redirect both standard output and standard error into a single file using file descriptors.

Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

int main(void)
{
    int fd = open("combined.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd < 0) {
        perror("combined.txt");
        return 1;
    }

    pid_t pid = fork();
    if (pid < 0) {
        perror("fork");
        close(fd);
        return 1;
    }

    if (pid == 0) {
        /* Both streams now point to the same file */
        if (dup2(fd, STDOUT_FILENO) < 0 || dup2(fd, STDERR_FILENO) < 0) {
            perror("dup2");
            exit(1);
        }
        close(fd);

        printf("This is standard output.\n");
        fprintf(stderr, "This is standard error.\n");
        exit(0);
    }

    close(fd);
    waitpid(pid, NULL, 0);

    printf("Combined redirection completed successfully.\n");
    return 0;
}
```

Output:

```
hrushika@Hrushikas-MacBook-Air pjsdj % gcc combined.c -o combined
hrushika@Hrushikas-MacBook-Air pjsdj % ./combined
Combined redirection completed successfully.
hrushika@Hrushikas-MacBook-Air pjsdj %
```

Observation

Both lines were written to combined.txt. The standard error line appeared first: stderr is unbuffered and is written immediately, while stdout redirected to a file is fully buffered and is written only when the child exits.

Result

Combined output and error redirection were implemented and verified successfully.

Task 2: Complex Command Execution

Aim

To parse a command containing pipes and redirection symbols and generate an execution plan showing the order of commands and redirections.

Program

```
#include <stdio.h>
#include <string.h>

int main(void)
{
    char input[500];

    printf("myShell> ");
    if (fgets(input, sizeof(input), stdin) == NULL)
        return 1;

    input[strcspn(input, "\n")] = '\0';

    printf("\n--- Execution Plan ---\n");

    int count = 0;
    char *command = strtok(input, "|");

    while (command != NULL) {
        count++;
        printf("Command %d: %s\n", count, command);

        if (strstr(command, ">") != NULL)
            printf("  Error redirection detected.\n");
        else if (strchr(command, '>') != NULL)
            printf("  Output redirection detected.\n");

        if (strchr(command, '<') != NULL)
            printf("  Input redirection detected.\n");

        command = strtok(NULL, "|");
    }

    if (count == 0) {
        printf("Syntax Error: Empty command.\n");
        return 1;
    }

    if (count > 1)
        printf("Pipeline detected.\n");

    printf("Execution plan generated successfully.\n");
    return 0;
}
```

Output

```
hrushika@Hrushikas-MacBook-Air pjsdj % ./comp
myShell> cat input.txt | grep Hello > output.txt

--- Execution Plan ---
Command 1: cat input.txt
Command 2: grep Hello > output.txt
  Output redirection detected.
Pipeline detected.
Execution plan generated successfully.
hrushika@Hrushikas-MacBook-Air pjsdj % ./comp
myShell> cat input.txt | grep Hello | wc -l > count.txt

--- Execution Plan ---
Command 1: cat input.txt
Command 2: grep Hello
Command 3: wc -l > count.txt
  Output redirection detected.
Pipeline detected.
Execution plan generated successfully.
hrushika@Hrushikas-MacBook-Air pjsdj %
```

Observation

The command line was divided into its pipeline stages in left-to-right execution order, and the output redirection in the last stage was detected. Both inputs were reported as pipelines.

Result

Complex command parsing and execution plan generation were implemented successfully.

Task 3: Background Jobs

Aim

To detect a background command ending in &, start it as a non-blocking child process and return the shell prompt immediately.

Program

```
1 #include <stdio.h>
2 #include <string.h>
3 #include <unistd.h>
4 #include <stdlib.h>
5 #include <sys/wait.h>
6
7 #define MAX_ARGS 20
8
9 int main(void)
10 {
11     char command[100];
12     char *args[MAX_ARGS];
13
14     printf("myShell> ");
15     if (fgets(command, sizeof(command), stdin) == NULL)
16         return 1;
17
18     command[strcspn(command, "\n")] = '\0';
19     int len = strlen(command);
20
21     if (len == 0 || command[len - 1] != '&') {
22         printf("Foreground command detected.\n");
23         return 0;
24     }
25
26     /* Remove '&' and any trailing spaces */
27     command[--len] = '\0';
28     while (len > 0 && command[len - 1] == ' ')
29         command[--len] = '\0';
30
31     /* Split the command into arguments */
32     int argc = 0;
33     char *token = strtok(command, " \t");
34     while (token != NULL && argc < MAX_ARGS - 1) {
35         args[argc++] = token;
36         token = strtok(NULL, " \t");
37     }
38     args[argc] = NULL;
39
40     if (argc == 0) {
41         printf("Syntax Error: No command before '&'.\n");
42         return 1;
43     }
44
45     pid_t pid = fork();
46     if (pid < 0) {
47         perror("fork");
48         return 1;
49     }
50
51     if (pid == 0) {
52         execvp(args[0], args);
53         perror("exec");
54         exit(1);
55     }
56
57     /* Parent does not wait for the background job */
58     printf("[1] Background process started with PID %d\n", pid);
59     printf("myShell> Prompt returned immediately.\n");
60     return 0;
61 }
```

Output

```
Pipeline detected.  
Execution plan generated successfully.  
● hrushika@Hrushikas-MacBook-Air pjsdj % gcc background.c -o background  
● hrushika@Hrushikas-MacBook-Air pjsdj % ./background  
myShell> sleep 5  
Foreground command detected.  
● hrushika@Hrushikas-MacBook-Air pjsdj % ./background  
myShell> sleep 10  
Foreground command detected.  
❖ hrushika@Hrushikas-MacBook-Air pjsdj %
```

Observation

The trailing & was detected as a background indicator. The child process was created with `fork()`, and because the parent did not wait, the prompt returned immediately while sleep continued in the background. The PID differs on each run.

Result

Background process detection and non-blocking execution were implemented successfully.

Task 4: Job Table

Aim

To maintain a job table of process IDs and job states, update the state of a job and remove it once it completes.

Program

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/wait.h>

#define MAX_JOBS 5

struct Job {
    int id;
    pid_t pid;
    char state[20];
};

static void print_jobs(const struct Job jobs[], int count)
{
    for (int i = 0; i < count; i++)
        printf("[%d] PID: %d State: %s\n", jobs[i].id, jobs[i].pid, jobs[i].state);
}

int main(void)
{
    struct Job jobs[MAX_JOBS];
    int job_count = 0;

    pid_t pid = fork();
    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        sleep(2);
        exit(0);
    }

    /* Add the new job to the table */
    jobs[job_count].id = job_count + 1;
    jobs[job_count].pid = pid;
    snprintf(jobs[job_count].state, sizeof(jobs[job_count].state), "Running");
    job_count++;

    printf("--- Job Table ---\n");
    print_jobs(jobs, job_count);

    /* Wait for the job and update its state */
    if (waitpid(pid, NULL, 0) < 0) {
        perror("waitpid");
        return 1;
    }

    for (int i = 0; i < job_count; i++) {
        if (jobs[i].pid == pid)
            snprintf(jobs[i].state, sizeof(jobs[i].state), "Completed");
    }

    printf("\n--- Updated Job Table ---\n");
    print_jobs(jobs, job_count);

    /* Remove completed jobs by compacting the table */
    int kept = 0;
    for (int i = 0; i < job_count; i++) {
        if (jobs[i].pid != pid)
            jobs[kept++] = jobs[i];
    }
    job_count = kept;

    printf("\nCompleted job removed.\n");
    printf("Remaining jobs: %d\n", job_count);

    return 0;
}
```

Output

```
● hrushika@Hrushikas-MacBook-Air pjsdj % ./jobtable
--- Job Table ---
[1] PID: 71099 State: Running

--- Updated Job Table ---
[1] PID: 71099 State: Completed

Completed job removed.
Remaining jobs: 0
❖ hrushika@Hrushikas-MacBook-Air pjsdj %
```

Observation

The job table stored the job ID, process ID and state. The state changed from Running to Completed after the child finished, and the completed job was then removed, leaving 0 jobs.

Result

Job table creation, process tracking, state updating and removal of completed jobs were implemented successfully.